

Senior AI Agent Engineer 2026 — интенсив с ИИ-тьютором

Программа состоит из 14 модулей, 136 практических промптов и сквозного production-проекта. Полный трек рассчитан на 502 часа; ускоренное прохождение с досрочной защитой модульных Gates — на 350–400 часов.

Формат обучения: AI-first code reading

Код пишет ИИ-тьютор. Студент не начинает с пустого файла и не тратит время на ручной boilerplate. Его работа ближе к современной роли Senior Engineer:

1. Понять теорию и задать вопросы до реализации.
2. Согласовать границы, упрощения и ожидаемое поведение.
3. Прочитать сгенерированный код и проследить один запрос end-to-end.
4. Объяснить, зачем нужен каждый существенный компонент.
5. Предсказать поведение при изменении входа или отказе зависимости.
6. Проверить diff, тесты, trace и метрики, а не доверять ответу модели.
7. Попросить изменить реализацию и оценить последствия изменения.
8. Защитить архитектурное решение и удалить неоправданную сложность.

Способность напечатать код по памяти не является Gate. Gate подтверждает, что студент понимает систему, способен направлять coding agent и отличает рабочую реализацию от правдоподобной, но неверной.

Как пользоваться ИИ-тьютором

1. Используй этот репозиторий как единый учебный monorepo.
2. В начале новой сессии отправляй мастер-промпт.
3. Запускай тему командой `начать <номер промпта>`.
4. Сначала тьютор создаёт конспект Markdown/PDF и даёт короткое объяснение.
5. Задавай вопросы; когда теория понятна, напиши `к практике`.
6. Тьютор сам реализует код и тесты, затем проводит guided code tour.
7. Используй команды `покажи поток запроса`, `разбери следующий файл`, `сломай пример`, `покажи diff` и `проверь моё объяснение`.
8. После темы тьютор обновляет конспект и PDF.
9. Не переходи к framework mapping до проверки from-scratch реализации.
10. В каждом модуле сохраняй ADR, eval report и git tag.

Ускоренное прохождение

- После короткой теории попроси показать architecture map и сразу переходи к agent-generated implementation, если тема уже знакома.
- Если artifact проходит acceptance criteria и fault tests, сокращай code tour до end-to-end trace, review diff и framework delta.

- Не пропускай модули 2, 6, 7, 12, 13 и 14; в них можно сократить чтение, но не implementation, code review и evidence.
- В модуле 8 поручай тьютору создавать micro-version каждого pattern, а глубоко production-test только три, выбранные benchmark.
- Экономия времени засчитывается только по executable evidence, а не по самооценке «я это уже знаю».

Мастер-промт

Ты — Principal AI Engineer, мой технический тьютор и coding agent. Я Senior Python Developer с DevOps/SRE-опытом: не объясняй базовый Python, Git, Docker, Kubernetes, SQL, OAuth и CI/CD, но AI-agent concepts объясняй с первого принципа и на конкретных примерах.

Каждую тему води в двух фазах. Фаза «Теория»: создай или обнови конспект в `notes/module_XX_topic/` в Markdown и PDF. Объясни проблему, ключевые понятия, один простой execution flow, ограничения и типичные ошибки. В чате дай компактное объяснение без огромных листингов и остановись для моих вопросов. Не переходи к коду, пока я не напишу `к практике`.

Фаза «Практика»: сначала покажи план и дерево файлов, затем самостоятельно реализуй рабочий typed Python-код и deterministic tests без agent frameworks. Я не пишу реализацию с пустого файла. После проверок проведи guided code tour: покажи один end-to-end trace, затем разбирай не больше одного-двух компонентов за ответ. Не вставляй полный код в чат — ссылайся на файлы и цитируй только ключевые фрагменты.

Проверяй моё понимание через вопросы на предсказание поведения, чтение diff, поиск намеренно добавленной ошибки и объяснение результатов тестов. Если я ошибся, вернись к конкретному участку кода и эксперименту. После разбора обнови конспект и PDF.

В реализации используй async-first typed Python, deterministic fakes, unit/contract/integration/property tests, deadline, cancellation, budgets и telemetry там, где они действительно нужны. Ищи retry hazards, duplicate side effects, races, prompt injection, schema drift, cross-tenant leakage и лишнюю agentic complexity.

Для изменчивых API используй первичные источники, указывай `verified_at`, spec/SDK version и known deprecations. Framework mapping показывай только после from-scratch реализации. Если agentic complexity не превосходит простой baseline, рекомендуй её удалить.

Ожидаемая структура ответа тьютора

1. Путь к созданному конспекту Markdown/PDF.
2. Короткая карта темы и простой execution flow.
3. Пауза для вопросов.
4. После `к практике`: план, дерево файлов и acceptance criteria.
5. Реализация тьютором и результаты тестов.
6. Guided code tour и end-to-end trace.
7. Fault/security experiments, dataset, baseline и метрики.
8. Проверка понимания по коду и diff.
9. Framework mapping после Gate.
10. Обновлённый конспект, production review и следующий шаг.

Как исполнять предметные промпты

Каждый пронумерованный промпт ниже — спецификация всей темы, а не одного ответа. Даже если в нём сразу описана реализация, тьютор обязан разделить работу:

- 1. Сначала выполнить только U.1: создать конспект, кратко объяснить тему и остановиться.
- 2. Ответить на вопросы по теории.
- 3. Только после команды `к практике` выполнить U.2–U.8 небольшими шагами.
- 4. Завершить тему через U.9–U.10 и обновить конспект.

Это правило имеет приоритет над формулировками отдельных предметных промптов. Команда «реализуй» всегда адресована тьютору, а не студенту.

Сквозной проект

Все модули развивают `Evidence & Change Agent`, который исследует versioned corpus и live systems, строит план, вызывает tools, создаёт evidence-backed report или patch, работает в sandbox, запрашивает scored approval, переживает crash и предоставляет trace/eval/audit.

Карта трека

Модуль	Часы	Артефакт
1. LLM Runtime	18	Provider-neutral Model Gateway
2. Tool Calling	28	Safe multi-provider Tool Runtime
3. Context Engineering	24	Context Compiler
4. Retrieval & Agentic RAG	32	Hybrid Knowledge Service
5. Memory Systems	28	Typed Hybrid Memory
6. Own Agent Framework	64	Agent Runtime v0–v5
7. Durable Graph Execution	32	Crash-safe Graph Executor
8. Advanced Agent Patterns	44	Pattern benchmark
9. Multi-Agent Systems	24	Supervisor/workers runtime
10. Protocol Engineering	24	MCP/A2A/AG-UI demo
11. Framework Internals	16	Comparative teardown и ADR
12. Security & High-Risk Agents	32	Sandbox и red-team suite
13. Evals, Observability & Optimization	40	Eval/release platform
14. Production Capstone	96	Production defense
Итого	502	

Модуль 1: LLM Runtime и Provider Abstraction

Цель — отделить probabilistic model от agent runtime и создать lossless adapter boundary. Это не обзор SDK, а исследование streaming, partial failure, capabilities, usage и inference economics.

Ключевые темы

1. Tokenization, prefill/decode, TTFT и throughput.
2. Streaming events и partial response.
3. Sampling, reasoning budget, stop/refusal.
4. Capability registry и fallback.
5. KV cache, prefix cache и prompt caching.
6. Record/replay и deterministic fake.

Набор промптов

Промпт 1.1: Model не является Agent

Начни только с фазы «Теория». На одном примере — обработка запроса пользователя с возможным вызовом tool — объясни разницу между model client, agent loop, workflow и distributed worker. Покажи одну общую схему и компактную сравнительную таблицу: что компонент знает, за что отвечает и какие ошибки возвращает наверх. Не выдавай интерфейсы и большой листинг. Создай конспект модуля 1 в Markdown/PDF и остановись для вопросов.

После моей команды `к практике` самостоятельно создай минимальный runnable demo без agent frameworks: четыре маленьких компонента, один happy-path trace и несколько boundary tests. Затем покажи дерево файлов и проведи code tour от входного запроса до результата, по одному-двум файлам за ответ.

Промпт 1.2: Provider-neutral contract

Сначала обнови конспект: на трёх коротких provider-response examples объясни, зачем нужны `ModelClient`, `ModelRequest`, `ModelResponse` и `ModelEvent`, что можно нормализовать, а что необходимо сохранить losslessly. Покажи error taxonomy и contract-test matrix без полного кода.

После `к практике` сам реализуй provider-neutral contracts и deterministic fake. Поддержи text, structured output, tool call, usage, refusal и streaming events, сохранив provider-specific blocks. Запусти contract tests, затем разбери со мной модели данных и один normal/failed event trace.

Промпт 1.3: Streaming и cancellation

Кратко объясни streaming lifecycle и обнови конспект одной временной диаграммой. После к практике самостоятельно расширь adapter и создай failure lab: stream оборвался, client отменил request, provider вернул 429/Retry-After, output завершился по token limit, usage отсутствует. Реализуй deadline/cancellation propagation и тесты. Затем пошагово покажи два trace: успешную отмену и ambiguous partial response. Вместе определим, какие retries безопасны, а какие меняют семантику.

Промпт 1.4: Capabilities и fallback

Сначала на понятном примере объясни capability negotiation и обнови конспект. После к практике сам реализуй registry без if model_name. Capability должна описывать tools, strict schema, modalities, context, streaming и preview features. Добавь router/fallback, который не может молча потерять требуемую capability, и property tests несовместимой fallback-модели. Затем покажи decision trace router и предложи мне предсказать результат для трёх конфигураций.

Промпт 1.5: Cache internals

Объясни KV cache, prefix cache и provider prompt caching через prefill/decode и добавь сравнение в конспект. После к практике сам создай и запусти эксперимент: stable prefix, изменение одного token, несколько tenants и sensitive content. Измерь hit rate, TTFT и cost, покажи таблицу результатов и код измерения. Затем попроси меня объяснить по trace, почему cache не является memory.

Промпт 1.6: Record/replay и evaluation

Сначала объясни record/replay на одном коротком trace и обнови конспект. После к практике сам реализуй adapter и deterministic fake для normal response, malformed output, tool call, partial stream, timeout и cancellation. Добавь JSONL runner, сохраняющий events, output, latency и usage. Запусти tests и разбери со мной один replay file. Заверши сравнением deterministic contract tests и repeated live evals.

Ожидаемый результат и Gate

- Два adapter проходят один contract suite.
- Provider metadata не теряется.
- Cancellation и deadline проходят через stream.
- Fallback проверяет capabilities.
- Есть воспроизводимый record/replay baseline.

Модуль 2: Structured Outputs и Native Tool Calling

Сначала строится provider-neutral tool protocol и безопасный executor. Затем один behavioral suite переносится на OpenAI Responses, Anthropic Tool Use и Gemini Function Calling.

Ключевые темы

1. JSON Schema и constrained generation.
2. Call/result correlation.
3. Parallel и streaming calls.
4. Retry, timeout, cancellation и idempotency.
- 5. Tool Registry и dynamic discovery.
6. Programmatic tool calling и deferred tool loading.
7. Policy, secrets и sandbox.

Набор промптов

Промпт 2.1: Tool protocol с нуля

После краткого объяснения сам реализуй `ToolSpec`, `ToolCall`, `ToolResult`, `ToolError` и `ToolExecutionPolicy`. Разбери `required/nullable`, `union/discriminator`, `bounds`, `additionalProperties`, `schema versioning` и `provider subsets`. Model output должен быть только предложением; execution начинается после `schema validation` и `authorization`.

Промпт 2.2: Registry и executor

Проведи review, затем расширь реализацию до `async ToolRegistry` и `ToolExecutor`. Registry хранит `stable name`, `version`, `schema hash`, `owner`, `risk`, `scopes` и `tenant visibility`. Executor поддерживает `deadline`, `cancellation`, `bounded concurrency`, `result-size limit`, `artifacts` и `progress`. Добавь `fake tools` и `property tests`.

Промпт 2.3: Parallel и streaming

Дай failure lab для streaming arguments: JSON невалиден до terminal event, calls приходят вперемешку, один branch завис, второй завершился. Требуй сборку по call ID, validation после завершения, dependency-aware execution, bounded concurrency и explicit join policy. Проверь partial success и cancellation всего run.

Промпт 2.4: Retry и side effects

Составь со мной retry matrix для transport, rate limit, timeout, validation, business error и unknown outcome. Сам реализуй idempotency key, deduplication и reconciliation для `send_email` и `charge_account`. Не принимай обещание exactly-once без доказанной transaction boundary.

Промпт 2.5: OpenAI Responses adapter

После Gate отобрази наш protocol на актуальный OpenAI Responses API: `function_call` → application execution → `function_call_output` с тем же `call_id`, strict schema, parallel и streaming calls. Проверь официальную документацию, зафиксируй дату/версию и прогони общий contract suite. SDK не должен заменить наш executor.

Промпт 2.6: Anthropic adapter

Реализуй mapping на Anthropic `tool_use / tool_result`, `tool_use_id`, несколько results в следующем user message, strict input и отдельный `output_config.format`. Добавь fine-grained streaming case с незавершёнными arguments. Сравни wire model и наш normalized contract.

Промпт 2.7: Gemini adapter

Реализуй mapping на Gemini `function_call / function_result`, `call_id`, parallel/compositional calls и opaque thought signatures. Любое сочетание tools со structured final output пропускай через capability gate. Зафиксируй preview/GA status и добавь несовместимую-model test.

Промпт 2.8: Tool security

Проведи threat model: malicious arguments/results, path traversal, command/SQL injection, SSRF, symlink attacks, secret exfiltration и schema rug pull. Сам реализуй `PolicyEngine`, per-action auth, secret broker, read/write capability split, egress allowlist и approval tier. Фильтр по словам запрещён.

Промпт 2.9: Dynamic discovery

Добавь dynamic discovery без автоматического доверия. Требуй owner/signature/schema hash, immutable tool snapshot на run и approval новой/изменённой capability. Сравни static registry, deferred loading, provider tool search и MCP. Измерь tool-selection accuracy и token cost при 10/100/1000 schemas.

Промпт 2.10: Programmatic tool calling

Реализуй provider-neutral bounded code-orchestration path: модель может написать программу, которая вызывает только allowlisted read-only tools, обрабатывает большие intermediate results и возвращает typed aggregate. Отдели program/tool call IDs, caller lineage, sandbox и budgets. Сравни direct sequential, parallel и programmatic calls по quality, evidence, tokens, latency и cost. Side effects и approval оставь вне программы.

Ожидаемый результат и Gate

- Три provider adapter проходят один behavioral suite.
 - Validation и authorization предшествуют execution.
 - Secrets не видны модели.
 - Parallel calls коррелируются независимо от порядка.
 - Fault suite подтверждает effectively-once в declared boundary.
-

Модуль 3: Context Engineering

Context — скомпилированный вход одного model call. Он не равен state, memory, checkpoint, RAG или cache.

Ключевые темы

1. Context window management, manifest и budgets.
2. Sliding window и position effects.
3. Prompt/semantic compression.
4. Summarization и drift.
- 5. Long-term context retrieval.
6. KV/prefix/provider caches.
7. Provenance и indirect injection.

Набор промптов

Промпт 3.1: Тахоному

Раздели context, canonical state, working/long-term memory, checkpoint, event log, RAG и cache. Для сквозного агента перечисли context sources. Каждый segment должен иметь source, trust, priority, freshness, TTL, token estimate и reason for inclusion. Сам оформи это как typed contracts и затем проведи со мной короткий code tour.

Промпт 3.2: Context Compiler

После объяснения сам реализуй `ContextSource`, `ContextCandidate`, `ContextManifest`, `ContextManager` и `PromptBuilder`. Compiler резервирует budgets, ранжирует по relevance/priority/trust/freshness, сжимает и объясняет inclusion. Prompt Builder принимает typed context, а не конкатенацию строк. Требуй snapshot tests.

Промпт 3.3: Window и summarization

Расширь проект sliding window, structured state extraction, rolling и hierarchical summary. Unresolved obligations, entity references и fact ledger храни отдельно от narrative. Создай 200-turn dataset и измерь recall, summary drift, token cost и lost-in-the-middle.

Промпт 3.4: Compression benchmark

Сравни extractive, prompt, semantic и tool-result compression. Требуй provenance links и verifier factual preservation. На budgets 4k/16k/64k сравни full context, sliding, summary, retrieval и hybrid. Построй quality-cost-latency frontier.

Промпт 3.5: Cache strategy

Проведи experiment по KV/prefix/provider prompt caching: stable prefix, изменение одного token, cache invalidation, tenant isolation и sensitive data. Требуй simulator и live benchmark с hit rate, TTFT и cost. Отдельно объясни context editing как compression, а не cache.

Промпт 3.6: Long-term retrieval

Сам реализуй извлечение context из state history, memory, knowledge и artifact stores. Hard tenant/ACL/trust filters применяются до relevance ranking. Manifest обязан объяснять каждое включение. Добавь cases: stale fact, contradiction, missing source и cross-user candidate.

Промпт 3.7: Context injection

Помести malicious instructions в document, tool result и memory. Context Compiler должен сохранить их как untrusted data, не повысить instruction priority и не дать изменить tool policy. Добавь attack metric и объясни, почему delimiter/sanitizer/classifier не являются достаточной security boundary.

Промпт 3.8: Senior review

Проведи architecture review Context Compiler по correctness, cost, cacheability, privacy и operability. Назови пять решений, которые сломаются на длинных runs или multi-tenancy. Сформируй P0-P2 refactoring plan и ADR выбора full context vs summary vs retrieval.

Ожидаемый результат и Gate

- Каждый model call имеет воспроизводимый `ContextManifest`.
- Стратегия window/compression выбрана экспериментально.
- Cache не смешивает tenants и не называется memory.
- Retrieved content не меняет authorization policy.

Модуль 4: Retrieval и Agentic RAG

До LlamaIndex реализуются ingestion, sparse/dense/hybrid retrieval, reranking, ACL и evaluation. Генератор не должен скрывать слабый retriever.

Ключевые темы

1. Ingestion/versioning/delete propagation.
2. BM25, dense vectors и ANN mental model.
3. Hybrid fusion, reranking и MMR.
4. Query rewrite/decomposition.
5. SQL, graph и API retrieval.
6. Provenance, citations, ACL и injection.

Набор промптов

Промпт 4.1: Ingestion lifecycle

Сам реализуй versioned ingestion pipeline: immutable source ID, original artifact, structural/semantic/parent-child chunks, metadata, dedup, incremental update, tombstones, embedding version, reindex и rollback. Добавь tests повторной доставки, partial failure и удаления из text/vector/cache layers.

Промпт 4.2: BM25 с нуля

Сам реализуй inverted index и BM25 без retrieval framework. Dataset должен включать identifiers, error codes, names и exact terms. Измерь Recall@k, MRR и nDCG. Затем проведи со мной complexity и correctness review созданного diff.

Промпт 4.3: Dense и ANN

Добавь embedding adapter и exact cosine search. Объясни HNSW/IVF trade-offs без переписывания production ANN library. Создай cases, где dense выигрывает и проигрывает. Зафиксируй model/version, normalization, dimension и re-embedding strategy.

Промпт 4.4: Hybrid и reranking

Сам реализуй Reciprocal Rank Fusion, weighted fusion, `Reranker` interface и MMR. Hard ACL/tenant/temporal filters применяются до semantic ranking. Сравни BM25, dense, hybrid и hybrid+reranker на одном dataset.

Промпт 4.5: Agentic retrieval

Добавь source classifier, query rewrite, multi-query, decomposition, parent-child retrieval и no-answer threshold. Создай ambiguous, stale, conflicting и multi-hop cases. Оцени retrieval trajectory отдельно от final generation и докажи uplift над retrieve-once baseline.

Промпт 4.6: SQL, Graph и API

Расширь KnowledgeService адаптерами SQL, knowledge graph и live API. Для SQL требуй read-only identity, schema linking, AST allowlist, EXPLAIN, timeout и row limit. Для graph покажи entity resolution и traversal. Нормализуй всё в evidence bundle.

Промпт 4.7: Citation и security

Реализуй citation verifier: claim связан с evidence span, source и version. Добавь malicious documents, ACL-protected chunks и deleted version. Метрики: citation correctness, groundedness, freshness, ACL leakage и injection success. Объясни, почему retrieval и generation quality измеряются отдельно.

Промпт 4.8: LlamaIndex mapping

Только после Gate повтори bounded pipeline через актуальные LlamaIndex retrievers, postprocessors, RouterQueryEngine и Workflows. Не используй legacy QueryPipeline. Сопоставь primitives с нашим service, прогони тот же suite и объясни, что framework скрывает.

Ожидаемый результат и Gate

- Versioned Hybrid Knowledge Service работает без agent framework.
- Hybrid сравнивается с простыми baselines.
- ACL применяется до выдачи evidence модели.
- Agentic retrieval принят только при измеримом эффекте.

Модуль 5: Memory Systems

Память проектируется как управляемая data system с write/read policy, ranking, consolidation и deletion, а не как vector store с chat history.

Ключевые темы

1. Working, short-term и long-term memory.
2. Semantic, episodic и procedural memory.
3. Vector и hybrid access.
4. Ranking и temporal validity.
5. Consolidation, eviction и forgetting.
6. Poisoning, consent и deletion.

Набор промптов

Промпт 5.1: Memory model

Сначала объясни multi-tenant память для агента, работающего месяцами, и обнови конспект. После к практике сам спроектируй систему из immutable events, derived records, write policy, retrieval/ranking, consolidation, conflict resolution, eviction и deletion. Полная chat history не должна автоматически становиться памятью.

Промпт 5.2: Record schema и write path

Сам определи record с tenant/subject/type/content, source-event IDs, valid time, confidence, importance, sensitivity, schema/embedding version, TTL и deletion state. Реализуй MemoryWriter с consent, classification, dedup, entity resolution, conflict detection и poisoning quarantine.

Промпт 5.3: Read path и ranking

Реализуй MemoryRetriever и MemoryRanker: hard ACL filters, sparse/ dense/structured candidates, relevance, recency, importance, confidence, diversity и temporal validity. Result объясняет причину retrieval. Добавь stale, contradictory и cross-tenant cases.

Промпт 5.4: Consolidation

Сам реализуй episode→semantic fact, trajectories→procedural memory и higher-level summary. Требуй provenance, confidence propagation и idempotent jobs. Измерь consolidation fidelity, contradiction и summary drift.

Промпт 5.5: Eviction и deletion

Сравни TTL, LRU, capacity и utility-based eviction. Реализуй tombstones, superseding, delete/export и propagation в text/vector/cache/backup layers. Добавь concurrent read/delete и embedding migration tests.

Промпт 5.6: Memory red-team

Инъецируй malicious instruction, fake preference, poisoned procedure, provenance spoofing и secret retention. Построй security dataset: cross-tenant leakage, unauthorized writes, poisoned activation и deletion failure. Определи stateless degraded mode.

Промпт 5.7: Memory evaluation

Спроектируй 100-session experiment: relevant recall/precision, contradiction, temporal correctness, downstream task uplift, token overhead, latency и storage. Сравни no-memory, recent-history, vector-only и hybrid. Не принимай memory без практического gain.

Промпт 5.8: Architecture defense

Проведи Senior review memory subsystem. Разбери consistency, multi-tenancy, data lifecycle, migration, observability и incident recovery. Назови пять причин отключить long-term memory в production и оформи ADR выбора store/ranking/consolidation policy.

Ожидаемый результат и Gate

- Собственные working/episodic/semantic/procedural stores.
- Write/read/ranking/consolidation/eviction реализованы отдельно.
- Нет observed cross-tenant leakage в versioned suite.
- Memory превосходит no-memory baseline либо отключается.

Модуль 6: Собственный Agent Framework

Центральный модуль. Запрещены LangGraph, LangChain Agents, LlamaIndex Agents/Workflows, CrewAI, AutoGen и Microsoft Agent Framework. Разрешены provider adapters, DB/queue clients, Pydantic, telemetry и testing.

Ключевые темы

1. Agent, ModelClient, Tool и ToolRegistry.
2. Planner и Executor.
3. Memory, ContextManager и PromptBuilder.
4. EventBus, State и reducers.
5. GraphExecutor и termination.
6. Retry, Reflection и CheckpointStore.
7. PolicyEngine, UsageLedger и Evaluator.

Набор промптов

Промпт 6.1: Framework RFC

Сначала объясни границы собственного async-first provider-neutral agent runtime. После к практике сам подготовь RFC. Обязательные компоненты: Agent, Tool, ToolRegistry, ModelClient, Planner, Executor, Memory, EventBus, ContextManager, PromptBuilder, State, GraphExecutor, RetryPolicy, Reflection, CheckpointStore, Evaluator, PolicyEngine и UsageLedger. Проведи review ownership, dependencies и запрета cyclic imports.

Промпт 6.2: Domain model

Сам определи Session, Thread, Run, Turn, Goal, Plan, Step, Action, Observation, Artifact, Event и Checkpoint. Раздели config и mutable state. Все canonical entities сериализуемы и versioned. Определи invariants и migrations, затем проверь models property tests.

Промпт 6.3: Event model

Сам реализуй typed EventBus и envelope: event/run/thread IDs, causation, correlation, scoped sequence, producer, schema version, payload и sensitivity. Версия этого модуля in-process. Раздели domain, telemetry и audit events. Проверь duplicate/out-of-order consumers и redaction.

Промпт 6.4: State и reducers

Реализуй typed state patches, optimistic version и reducer per field: replace, append, merge-by-ID, max/min и custom. Попроси меня доказать associativity/commutativity/idempotence там, где parallel merge на них опирается. Добавь conflicts, replay и schema migration tests.

Промпт 6.5: Planner с нуля

Сначала объясни planner, затем сам реализуй deterministic operator/HTN planner с preconditions, effects и dependency graph. Затем разреши LLM только предлагать typed Plan. Compiler проверяет feasibility, capabilities, risk, budgets и completion predicate. Добавь invalid/cyclic/stale plan tests.

Промпт 6.6: Executor

Реализуй Executor, который выбирает ready steps, повторно проверяет preconditions/authorization, резервирует budget, выполняет независимые actions с bounded concurrency, сохраняет artifact/observation и принимает continue/replan/reflect/wait/complete/fail. Проверь cancellation и partial completion.

Промпт 6.7: Agent loop

Собери явную state machine INIT→BUILD_CONTEXT→MODEL→VALIDATE→AUTHORIZE→EXECUTE→OBSERVE. Добавь branches REPLAN/REFLECT/WAIT/COMPLETE/FAIL. Каждый transition имеет guard и typed I/O. Termination определяется completion predicate, deadline, step/token/cost budgets и loop detector, а не фразой модели.

Промпт 6.8: Retry и Reflection

Реализуй error classification и bounded RetryPolicy. Reflection оформи как typed critique с failure type, evidence IDs, violated constraint, proposed change и confidence. Она не вызывает tools и имеет отдельный budget. Сравни no-reflection/one-reflection/external-verifier и удали reflection без uplift.

Промпт 6.9: Checkpoint и replay

Сам реализуй atomic checkpoint: runtime/state version, cursor, ready/completed sets, approvals, event offset и usage. Секреты и open connections не сериализуются. Реализуй live/recorded/hybrid replay и fork с новой lineage. Инъектируй crash до/после state commit.

Промпт 6.10: Integration и Senior defense

Собери framework v0-v5, не переписывая компоненты. Запусти две разные прикладные задачи без изменения core. Проведи architecture defense по correctness, API stability, testability, security hooks, cost и operability. Выдай P0-P2 refactoring и сравни с простым tool loop.

Ожидаемый результат и Gate

- Все обязательные компоненты реализованы без agent framework.
 - Две задачи используют одно ядро.
 - Deterministic fake полностью заменяет модель в tests.
 - Crash/replay и budgets имеют наблюдаемую семантику.
 - Каждое решение model→policy→tool видно в trace.
-

Модуль 7: Durable Graph Execution и LangGraph Teardown

Single-process runtime превращается в durable scheduler/workers. Только после этого изучается LangGraph.

Ключевые темы

1. Directed graphs, cycles и termination.
2. Fan-out/fan-in и join semantics.
3. Durable ready queue, leases и fencing.
4. Checkpoint/event log/replay.
5. Side-effect boundaries.
6. Interrupt, HITL и time travel.
7. Workflow/schema migration.

Набор промптов

Промпт 7.1: Formal execution model

Сначала объясни, почему graph с cycles не DAG, на маленьком примере. Затем сам зафиксируй formal model: State, Node, Edge, Guard, Reducer, Scheduler, ReadyQueue, JoinPolicy, Interrupt и RunBudget. Дай counterexamples для non-associative reducer, unbounded cycle и nondeterministic replay.

Промпт 7.2: Graph Executor

Сам расширь наш GraphExecutor: conditional edges, cycles, fan-out, all/any/quorum join, subgraphs и deterministic test scheduler. Compiler ловит unreachable nodes, reducer mismatch и invalid joins. Добавь step/deadline/token/cost termination.

Промпт 7.3: Durable scheduler/workers

Отдели scheduler от workers, добавь durable ready queue, lease, heartbeat и fencing. Fencing работает только если resource атомарно проверяет monotonic token. Инъектируй duplicate delivery, stale worker, lost heartbeat и два scheduler. Определи ownership state.

Промпт 7.4: Side effects

На примерах email/payment разметь pure computation, activity и side effect. Спроектируй outbox/inbox, idempotency, dedup, reconciliation и compensation. Инъектируй crash после внешнего эффекта до checkpoint. Определи effectively-once business boundary и residual risk.

Промпт 7.5: Interrupt и time travel

Реализуй typed interrupt/resume contract, approval expiry, cancellation race и branch fork. Time travel не перезаписывает историю и не повторяет внешний effect без policy. Добавь recorded/live replay boundary и lineage visualization.

Промпт 7.6: Version migration

Сам обнови graph/state/tool schema при незавершённых runs. Сравни pin-old-code, migrate checkpoint и compatibility adapter. Добавь expand/migrate/contract strategy, rollback и tests несовместимой версии.

Промпт 7.7: LangGraph mapping

Теперь сопоставь наш runtime с актуальным LangGraph v1: StateGraph, reducers, nodes/edges/Command, checkpointer, Store, interrupts, subgraphs, streaming и time travel. Для каждого укажи, что framework скрывает и какие side-effect/security guarantees остаются приложению.

Промпт 7.8: Equivalence и build-vs-buy

Прогони один behavioral/fault suite на собственном executor и LangGraph. Сравни также general-purpose durable engine класса Temporal/DBOS. Подготовь ADR по determinism, migration, operations, lock-in и стоимости. Custom scheduler не должен автоматически стать production choice.

Ожидаемый результат и Gate

- Kill/restart не теряет run.
- Duplicate delivery не дублирует operation в declared boundary.
- Parallel merge имеет явную algebra.
- Незавершённый run переживает schema migration.
- LangGraph выбран или отвергнут через ADR.

Модуль 8: Advanced Agent Patterns

Все паттерны реализуются под единым interface на одном dataset. Сначала baseline, затем from-scratch pattern, failure analysis, framework mapping и ablation.

Универсальный префикс к каждому промπτу

Не начинай с пересказа статьи. Сначала попроси меня определить baseline и измеримую проблему. Для паттерна раскрой state/transition model, дополнительные model calls, data flow, termination, budgets, failure modes и security. Дай from-scratch задание, framework mapping и eval. Признай полезность только при practically meaningful gain.

Набор промπτов

Промпт 8.1: ReAct

Примени универсальный префикс к ReAct. Реализуй THINK→VALIDATE→ACT→OBSERVE без требования hidden chain-of-thought. Добавь loop detector, sanitized observation и completion predicate. Сравни с direct tool-call baseline.

Промпт 8.2: Plan-and-Execute

Разбери typed plan, preconditions, dependency scheduler, verifier, approval и replanning triggers. Сравни с ReAct на stale-plan cases и измерь plan validity, replans и critical-path latency.

Промпт 8.3: ReWOO

Реализуй planner с #E1... evidence variables, compiler ссылок, dependency workers и final solver. Инъектируй broken reference и wrong observation. Измерь model turns и parallel speedup.

Промпт 8.4: Self-Correction

Реализуй GENERATE→VERIFY→DIAGNOSE→PATCH только для задач с внешним verifier: tests, compiler, schema или rule engine. Измерь first/final pass, introduced regressions и repair cost.

Промпт 8.5: Self-Reflection

Реализуй critic с rubric и evidence IDs, затем accept/revise/escalate. Critic не вызывает side-effect tools. Калибруй его против humans и сравни с обычным дополнительным generation call.

Промпт 8.6: Reflexion

Реализуй actor→feedback→evaluator→reflection writer→vetted episodic/ procedural memory. Добавь provenance, TTL и poisoning checks. Измерь learning curve, negative transfer и memory overhead.

Промпт 8.7: Self-Discover

Создай registry reasoning modules, select/adapt/compose typed scaffold и execute. Логируй selection. Сравни stability, success и tokens с одним фиксированным scaffold.

Промпт 8.8: Tree of Thoughts

Реализуй generator, value function, BFS/DFS/beam, dedup, depth и budget. Не выполняй необратимые effects внутри search. Измерь success относительно explored nodes/tokens и evaluator accuracy.

Промпт 8.9: Graph of Thoughts

Добавь thought graph с provenance и operations generate/score/refine/ aggregate, merge/dedup и graph budget. Проведи ablation operations и сравни quality-cost с ToT.

Промпт 8.10: Hierarchical Planning

Реализуй goal tree, milestone contracts, recursive decomposition с max depth, bottom-up validation и progress roll-up. Инъектируй ошибку верхнего уровня и измерь scope replanning.

Промпт 8.11: Multi-Planner

Параллельно создай diverse candidate plans, normalize, проверь feasibility и выбери/synthesize через arbiter. Измерь diversity, oracle gap, arbiter accuracy и cost; majority vote не считать истиной.

Промпт 8.12: Multi-Executor

Реализуй capability dispatch, bounded worker pool, leases, dependency scheduling, artifact ownership, cancellation и join. Измерь speedup, duplicate work, conflicts и partial-failure behavior.

Промпт 8.13: Deep Agents

Собери long-horizon harness: planning, filesystem workspace, skills, subagents, context compaction, artifacts, sandbox, checkpoint и approvals. Объясни, почему это composition, а не новый reasoning algorithm. Затем сравни с актуальным Deep Agents implementation.

Промпт 8.14: Pattern tournament

Запусти direct, deterministic workflow, ReAct и все patterns на frozen dataset. Собери success, variance, steps, latency, tokens, cost и policy violations. Построй Pareto frontier и decision rubric. Удали patterns, которые не дают meaningful effect.

Ожидаемый результат и Gate

- Все 13 обязательных patterns имеют from-scratch micro-implementation.
 - Для каждого зафиксированы when/pros/cons/failures/framework mapping.
 - Один общий benchmark исключает cherry-picking.
 - Три выбранных patterns проходят глубокий fault/security suite.
-

Модуль 9: Multi-Agent Systems без театра агентов

Цель — научиться доказывать необходимость нескольких агентов, а затем строить coordination runtime с явными contracts, ownership и termination. Несколько personas в одном prompt не считаются multi-agent системой.

Ключевые темы

1. Single-agent baseline и decomposition boundaries.
2. Supervisor, handoff, router, peer-to-peer и blackboard.
3. Message envelope, mailbox, directory и capability discovery.
4. Delegation depth, leases, budgets и termination detection.
5. Shared workspace, ownership, merge и conflict resolution.
6. Identity propagation, least privilege и confused deputy.
7. Multi-agent eval и ablation.

Набор промптов

Промпт 9.1: Когда multi-agent оправдан

Дай мне пять задач, где multi-agent выглядит убедительно, но только в части случаев действительно нужен. Я должен выбрать simplest viable architecture: один model call, deterministic workflow, один agent с tools, subworkflow или multi-agent. Проверяй решение через isolation boundary, parallelism, heterogeneous permissions/models и independent lifecycle. Для каждого выбора потребуй falsifiable hypothesis и baseline.

Промпт 9.2: Message protocol с нуля

Сам реализуй typed AgentId, MessageEnvelope, CorrelationId, CausationId, Mailbox, AgentDirectory и DeliveryReceipt. Нужны request/reply/event, schema version, deadline, priority, trace context и idempotency key. Добавь duplicate, out-of-order, poison message и expired deadline tests. Не используй готовый multi-agent framework.

Промпт 9.3: Supervisor и capability-scoped delegation

Спроектируй supervisor, который декомпозирует goal в bounded tasks, выбирает worker по typed capability, делегирует только нужные tools и budget, проверяет artifact contract и умеет отменить subtree. Не передавай worker весь parent context. Проведи review на unbounded delegation, retry amplification и confused deputy.

Промпт 9.4: Handoff и conversational ownership

Реализуй handoff как явную передачу ownership, а не как магическую смену system prompt. Определи preconditions, context projection, pending obligations, user-visible transition, rollback и audit. Смоделируй rejected handoff, cycle A→B→A, потерянное сообщение и отмену пользователем.

Промпт 9.5: Shared workspace и concurrency

Сам реализуй blackboard/workspace с immutable artifacts, optimistic versioning, leases и merge policy. Два worker одновременно редактируют plan и один падает после external effect. Требуй conflict tests, compensation decision и доказательство отсутствия silent last-write-wins.

Промпт 9.6: Liveness и budget algebra

Формализуй termination: max delegation depth, global/per-agent step budget, wall-clock deadline, token/cost budget, cycle detection и quiescence. Проверь deadlock, livelock, orphan worker, retry storm и supervisor crash. Потребуй metrics для useful work, coordination overhead и budget attribution.

Промпт 9.7: Multi-agent security review

Построй threat model для supervisor/workers: spoofed identity, privilege laundering, malicious artifact, prompt injection через mailbox, cross-agent data leak и compromised worker. Сам реализуй signed/scoped delegation token, policy check на каждом hop, provenance и revocation test.

Промпт 9.8: Framework mapping

После прохождения Gate отобрази наши primitives на LangGraph, OpenAI Agents SDK handoffs, LlamaIndex AgentWorkflow, Microsoft Agent Framework и CrewAI. Если provider предлагает native multi-agent beta, добавь его как отдельный эксперимент, а не assumed baseline. Для каждого покажи canonical state, delivery guarantees, checkpoint boundary, permission model и termination. Отдельно отметь, что framework скрывает и что остаётся ответственностью приложения.

Промпт 9.9: Multi-agent ablation

Запусти frozen dataset на четырёх вариантах: direct call, deterministic workflow, single agent и multi-agent. Сравни task success, tail latency, tokens, cost, coordination steps, policy violations и recovery. Используй повторные запуски и confidence intervals. Multi-agent принимается только при измеримом выигрыше или необходимой security/isolation boundary.

Ожидаемый результат и Gate

- Runtime поддерживает supervisor, handoff и bounded parallel workers.
- Identity, budget, deadline и trace context сохраняются на каждом hop.
- Duplicate/out-of-order/cycle/crash сценарии покрыты тестами.

- Multi-agent вариант либо выигрывает у baseline, либо удалён из проекта.
-

Модуль 10: Protocol Engineering — MCP, A2A и AG-UI

Цель — реализовать протоколы на wire level и только после этого подключить SDK. Эти протоколы решают разные задачи: agent↔tools/context, agent↔agent и agent-backend↔interactive UI.

Ключевые темы

1. Capability discovery, negotiation и versioning.
2. MCP tools, resources, prompts, roots, sampling и elicitation.
3. MCP stdio и Streamable HTTP, progress и cancellation.
4. A2A Agent Card, Message, Task, Part, Artifact и streaming.
5. A2A authorization, long-running tasks и push notifications.
6. AG-UI events, shared state, interrupts и reconnect.
7. Conformance, fuzzing, compatibility и protocol security.

Набор промптов

Промпт 10.1: Protocol boundaries

Сравни MCP, A2A и AG-UI не по маркетингу, а по trust boundary, discovery, transport, state ownership, lifecycle, auth и failure semantics. Дай десять integration scenarios; я должен выбрать протокол или объяснить, почему достаточно обычного HTTP/event stream. Исправляй попытки заменить application architecture протоколом.

Промпт 10.2: MCP stdio с нуля

По актуальной MCP specification сам напиши минимальные client/server поверх stdio без SDK: initialize/negotiation, `tools/list`, `tools/call`, structured error и shutdown. Используй JSON-RPC framing строго по spec, golden packet fixtures и неизвестный method/version tests. Не добавляй network auth к локальному stdio.

Промпт 10.3: MCP capabilities и Streamable HTTP

Расширь реализацию resources/prompts и только релевантными roots, sampling/elicitation capabilities. Затем добавь Streamable HTTP, session semantics, progress, cancellation, reconnect и authorization. Проведи threat review: origin validation, SSRF, DNS rebinding, session hijack, confused deputy и запрет token passthrough.

Промпт 10.4: A2A core с нуля

По актуальной A2A 1.0 specification реализуй Agent Card discovery, Message, Task, Part, Artifact, SendMessage и streaming path без SDK. Зафиксируй task state machine, context/task correlation и supported interfaces. Создай packet fixtures и state-transition tests; не выдумывай endpoint или method из старого draft.

Промпт 10.5: A2A long-running и security

Добавь resumable long-running task, input-required/auth-required path, cancellation, reconnect и push notification там, где они оправданы. Спроектируй auth из Agent Card metadata без доверия к самой карточке. Проверь replay, forged card, artifact tampering, duplicate request, expired credential и tenant isolation.

Промпт 10.6: AG-UI event runtime

Реализуй минимальный event-based bridge между agent backend и UI: run lifecycle, text/tool events, state snapshot/delta, interrupt, approval и reconnect. Определи ordering, deduplication, backpressure и user cancellation. Зафиксируй, какие events stable, draft или deprecated в выбранной версии. UI не должен напрямую получать секреты tool runtime или становиться canonical owner execution state.

Промпт 10.7: End-to-end interoperability

Собери demo: UI через AG-UI запускает local agent, тот получает repository tool через MCP и делегирует bounded research task удалённому агенту через A2A. Протяни trace, identity, deadline, cancellation и provenance end-to-end. Инъектируй падение каждого компонента и покажи, какой слой отвечает за recovery.

Промпт 10.8: Conformance и version migration

Создай protocol conformance suite: golden packets, schema validation, malformed/oversized frames, unknown fields, downgrade, timeout, cancellation race и fuzz/property tests. Затем спроектируй migration между двумя версиями без flag day. SDK подключи только после сравнения его wire trace с нашими fixtures.

Ожидаемый результат и Gate

- Есть минимальные реализации MCP, A2A и AG-UI без SDK.
- Wire fixtures соответствуют зафиксированным версиям спецификаций.
- Auth, identity, trace, deadline и cancellation проходят end-to-end.
- Студент умеет обосновать, когда протокол вообще не нужен.

Модуль 11: Framework Internals и Build-vs-Buy

Цель — не «выучить пять фреймворков», а прогнать один bounded use case и понять, какие собственные primitives они заменяют, какие guarantees дают и какие риски добавляют.

Ключевые темы

1. LangGraph durable state graph.
2. OpenAI Agents SDK: loop, tools, handoffs, guardrails и tracing.
3. LlamaIndex Workflows/agents и data-centric composition.
4. Microsoft Agent Framework agents/workflows/harness.
5. CrewAI Flows/Crews как контролируемое сравнение.
6. Migration risk, lock-in, extension points и observability.

Набор промптов

Промпт 11.1: Teardown rubric

Составь framework teardown rubric: canonical state, scheduler, loop, persistence, interrupt, retries, side effects, streaming, context, memory, tool schema, multi-agent, tracing, testability, extension points, versioning и lock-in. Для каждого критерия потребуй executable probe, а не пересказ документации.

Промпт 11.2: LangGraph teardown

Перенеси один crash-safe flow из нашего Graph Executor в актуальный LangGraph. Сопоставь state, node, edge, command/routing, checkpoint, interrupt и replay. Проведи crash-before/after-effect tests и сравни trace с reference runtime. Отметь semantic differences, которые нельзя скрыть adapter.

Промпт 11.3: OpenAI Agents SDK teardown

Реализуй тот же bounded use case в актуальном OpenAI Agents SDK. Разбери loop termination, tool calls, handoffs, guardrails, sessions/context и tracing. Используй provider abstraction там, где это реально возможно, и явно зафиксируй provider coupling. Сравни failure ownership с нашим runtime.

Промпт 11.4: LlamaIndex teardown

Реализуй data-heavy вариант через актуальные LlamaIndex agents/ workflows. Исследуй event model, state, retrieval/data abstractions, human-in-the-loop и multi-agent composition. Не принимай встроенную memory за доказательство correctness: прогоняй наши retrieval, deletion, checkpoint и tenant-isolation tests.

Промпт 11.5: Microsoft Agent Framework и CrewAI

Проведи controlled comparison Microsoft Agent Framework и CrewAI на одном scenario. Отдельно проверь deterministic workflow против autonomous crew, а также Microsoft Agent Framework Harness против нашего long-horizon harness. Сравни durability, approvals, context compaction, memory, telemetry и deployment model. Старые AutoGen/Semantic Kernel tutorials используй только как migration material; сначала проверь актуальный статус и официальную migration path.

Промпт 11.6: Black-box fault probes

Для всех framework-вариантов запусти одинаковые probes: malformed tool args, duplicate result, timeout, cancellation, context overflow, checkpoint corruption, process crash и incompatible state migration. Заполни evidence matrix «documented / observed / application-owned».

Промпт 11.7: Architecture Decision Record

Напиши со мной ADR build-vs-buy. Варианты: own runtime, LangGraph, OpenAI Agents SDK, LlamaIndex, Microsoft Agent Framework, CrewAI и hybrid. Оцени fit, guarantees, ecosystem, operability, security, portability, migration cost и team skill. Выбери framework только для конкретного production boundary, не «лучший вообще».

Ожидаемый результат и Gate

- Один behavioral/fault suite работает на всех кандидатах.
- Есть source-level и black-box mapping внутренних primitives.
- Framework выбран через ADR, а не по популярности.
- Собственный runtime остаётся executable specification и test oracle.

Модуль 12: Security и High-Risk Agents

Цель — проектировать agent как потенциально скомпрометированный decision component. Ни model output, ни retrieved content, ни remote tool/agent не являются доверенными инструкциями.

Ключевые темы

1. Assets, trust boundaries и agent-specific abuse cases.
2. Direct/indirect prompt injection и instruction/data separation.
3. Identity, delegated authorization, approvals и audit.
4. Tool broker, sandbox, egress и secret isolation.
5. Memory/RAG poisoning и provenance.
- 6. MCP/A2A/tool supply chain.
7. Code, browser, computer-use и voice agents.
8. Red teaming, incident response и safe degradation.

Набор промптов

Промпт 12.1: Threat model как executable artifact

Построй threat model для Evidence & Change Agent: assets, actors, entry points, data flows, trust boundaries и abuse cases. Используй актуальные OWASP agentic/LLM risks как checklist, но добавь system-specific threats. Для каждого риска свяжи preventive control, detective control, test, telemetry, owner и residual risk.

Промпт 12.2: Prompt injection lab

Создай adversarial corpus с direct/indirect injection в web page, issue, PDF metadata, code comment, tool result и memory. Реализуй typed instruction provenance, context zoning, capability gating и output/effect validation. Не выдавай «sanitize prompt» за защиту. Измерь attack success rate и utility loss на benign dataset.

Промпт 12.3: Identity и delegated authorization

Спроектируй identity chain user→agent→subagent→tool. Каждый hop получает audience-bound, time-bound и task-bound capability без передачи исходного bearer token. Добавь policy decision/enforcement points, step-up approval, revocation и audit. Проверь confused deputy, privilege escalation, replay и approval spoofing.

Промпт 12.4: Tool broker и sandbox

Реализуй out-of-process tool broker: allowlisted command/API, typed args, filesystem scope, network egress policy, resource quota, deadline, output limit, secret broker и immutable audit. Sandbox считай одним defense layer, а не security boundary по умолчанию. Проведи escape, symlink/path traversal, DNS/SSRF и fork/resource bomb tests.

Промпт 12.5: Safe code agent

Сам построй code-change flow: untrusted repo checkout, isolated worktree, dependency policy, static/secret scan, tests, diff-only artifact, human approval и separate deploy identity. Агент не имеет production credential. Смоделируй malicious test, poisoned package, prompt injection в repository и destructive command.

Промпт 12.6: Browser, computer-use и voice

Сравни browser DOM/API automation, screenshot computer-use и voice agent по observability и risk. Спроектируй transaction classifier, domain/action allowlist, visible intent confirmation, anti-phishing, DTMF/PII handling и post-condition verification. Любые финансовые, legal, account или destructive effects требуют отдельного approval path.

Промпт 12.7: Memory и supply-chain poisoning

Атакуй memory, vector index, MCP server metadata, tool description, Agent Card и remote artifact. Реализуй signed provenance, trust score, quarantine, write validation, tenant ACL, deletion propagation и dependency/version pinning. Проверь delayed activation и poisoning, переживающий compaction/consolidation.

Промпт 12.8: Red-team campaign

Спроектируй repeatable red-team suite из attack taxonomy, mutation generator, deterministic fixtures и live adversarial runs. Метрики: attack success, unsafe effect, detection, time-to-containment, false positive и utility. Для найденного critical проведи incident exercise: revoke, isolate, preserve evidence, notify, patch и regression test.

Промпт 12.9: Security release review

Проведи formal go/no-go review. Требуй data-flow diagram, threat register, policy-as-code tests, sandbox evidence, least-privilege proof, secrets inventory, audit integrity, abuse monitoring, incident runbook и residual risk acceptance. Не разрешай release только потому, что model прошла content moderation.

Ожидаемый результат и Gate

- Threat model связан с исполняемыми security tests.
- Любой side effect проходит policy, scoped authorization и audit.
- Untrusted content не расширяет capabilities агента.
- Нет открытых critical/high без явного risk acceptance.

Модуль 13: Agent Evals, Observability и Optimization

Цель — превратить «кажется, агент стал лучше» в воспроизводимый release process. Оцениваются final outcome, trajectory, effects, policy compliance, recovery и economics.

Ключевые темы

1. Eval pyramid, frozen datasets и counterfactual baselines.
2. Outcome, trajectory, tool, retrieval, memory и security metrics.
3. Repeated trials, variance и statistical uncertainty.
4. Human labels и calibrated LLM-as-judge.
5. OpenTelemetry traces, logs, metrics и privacy.
- 6. Online eval, canary, SLO и regression gates.
7. Cost/latency budgets, model routing и caching.
8. Prompt/policy optimization без benchmark leakage.

Набор промптов

Промпт 13.1: Eval architecture

Спроектируй eval platform: versioned task spec, immutable dataset, fixtures, runner, trace store, graders, experiment metadata и report. Раздели unit/contract, component, trajectory, end-to-end, adversarial и online evals. Для каждого слоя зафиксируй owner, cadence, cost и release gate.

Промпт 13.2: Dataset engineering

Построй dataset для Evidence & Change Agent: normal, boundary, ambiguous, impossible, stale, permission-denied и adversarial cases. Добавь difficulty strata, provenance, temporal split, contamination checks и annotation guide. Создай direct-call и deterministic-workflow baselines до изменения agent architecture.

Промпт 13.3: Trajectory grading

Реализуй event-derived graders: task success, evidence correctness, tool precision/recall, invalid call, unnecessary step, recovery, duplicate effect, policy violation и budget adherence. Отдели плохой model decision от tool/runtime failure. Проверь graders на synthetic known-good/known-bad traces.

Промпт 13.4: LLM-as-judge calibration

Создай rubric с atomic criteria и blind pairwise comparison. Собери human-labeled calibration set, измерь agreement, bias к verbosity/ position/style и sensitivity к prompt injection в candidate output. Используй judge только после calibration; disagreement и low confidence направляй человеку.

Промпт 13.5: Repeated experiments и статистика

Для stochastic agent запусти repeated trials с фиксированными versioned configs. Покажи distribution, confidence interval и effect size, а не один average. Учти paired design, multiple comparisons и tail failures. Потребуй practical significance threshold до принятия изменения.

Промпт 13.6: OpenTelemetry instrumentation

Спроектируй traces вокруг model, tool, retrieval, memory, planner, checkpoint, protocol hop и approval. Протяни trace/span context через workers, MCP и A2A. Зафиксируй semantic conventions/version, redaction, sampling и cardinality limits. Trace должен восстанавливать trajectory, но не раскрывать secrets или полный sensitive prompt.

Промпт 13.7: Fault и security evaluation

Подключи deterministic fault proxy: latency, 429, partial stream, malformed schema, stale retrieval, duplicate tool result, worker crash, checkpoint corruption и protocol disconnect. Объедини с prompt injection/privilege suite. Измерь graceful degradation, recovery time, unsafe effect и audit completeness.

Промпт 13.8: Online SLO и canary

Определи SLI/SLO: task success proxy, unsafe-effect rate, p95/p99 latency, cost/task, escalation, abandonment и recovery. Спроектируй shadow, canary, feature flag, rollback и kill switch. Online feedback не должен автоматически становиться trusted memory или training label.

Промпт 13.9: Cost и model routing

Построй per-step usage ledger и Pareto analysis quality/latency/cost. Реализуй capability-aware router, small-model first, escalation, deterministic shortcut, stable-prefix caching и concurrency limits. Проверь quality regression по strata и запрети fallback, теряющий required capability или safety property.

Промпт 13.10: Optimizer с нуля, затем framework

Реализуй простой offline optimizer: candidate generator, train/dev/test split, objective с quality/cost/safety constraints, early stopping и experiment ledger. Защити от test leakage и overfitting. Только затем сравни подход с актуальными DSPy/GEPA или другими optimization tools на том же protocol и удержанном test set.

Промпт 13.11: Release decision

Подготовь release scorecard: baseline deltas, confidence, regressions по strata, security/fault results, cost capacity, SLO budget, known limits и rollback evidence. Проведи со мной adversarial review: я защищаю release, ты играешь Staff evaluator, Security и SRE. Решение должно быть воспроизводимо из artifacts.

Ожидаемый результат и Gate

- Любое архитектурное изменение сравнивается с frozen baseline.
 - Есть calibrated outcome/trajectory/security graders.
 - Trace объясняет решение, эффекты, recovery и стоимость.
 - Release блокируется автоматически при regression или policy breach.
-

Модуль 14: Production Capstone и Senior Defense

Цель — вывести ограниченную, но настоящую agentic систему в production под контролем. Capstone не обязан использовать все patterns: зрелость проявляется в удалении ненужной автономности.

Product brief

Evidence & Change Agent принимает versioned engineering request, исследует репозиторий и разрешённые live sources, строит evidence-backed план, предлагает patch или runbook, выполняет только scoped reversible actions, запрашивает approval перед high-impact effect, переживает crash и оставляет полный audit/eval trace.

Обязательное ядро:

1. Два model adapters и capability-aware routing.
2. Safe tool runtime, registry, timeout/retry и sandbox.
3. Context compiler, hybrid retrieval и typed memory.
4. Planner/executor/verifier, durable graph и checkpoint/replay.
5. Idempotent effects, approval и compensation policy.
6. Threat model, red-team suite и delegated authorization.
7. OpenTelemetry, offline/online eval и release gate.
- 8. Один ограниченный MCP, A2A или AG-UI integration path.

Multi-agent, Reflexion, ToT/GoT и другие patterns добавляются только после ablation с measurable benefit.

Набор промптов

Промпт 14.1: RFC и scope

Помоги мне написать production RFC: user/problem, non-goals, threat model summary, simplest baseline, agentic justification, architecture, canonical state, trust boundaries, data retention, SLO, eval plan, rollout и rollback. Задавай вопросы как Principal review board и отклоняй vague requirements.

Промпт 14.2: Walking skeleton

Сам реализуй один thin vertical slice от API/UI до model, одного read-only tool, evidence artifact и trace. Без multi-agent и reflection. Требуй hermetic local environment, deterministic test, deployment manifest, health/readiness и minimal dashboard. Сначала докажи end-to-end state ownership.

Промпт 14.3: Durable execution и effects

Расширь slice до crash-safe graph: checkpoint, resume, versioned state, idempotency ledger, outbox/inbox, approval interrupt и cancellation. Инъектируй crash до/после каждого external effect и process upgrade посередине run. Я должен доказать at-most-once business effect или корректную reconciliation.

Промпт 14.4: Knowledge, context и memory

Подключи hybrid retrieval с ACL/provenance, context compiler и typed memory. Проведи freshness, deletion, cross-tenant, poisoning и context overflow tests. Покажи ablation no-RAG/no-memory/no-compression и объясни, какие данные являются source of truth, а какие лишь hint.

Промпт 14.5: Security readiness

Проведи полный threat-model review и adversarial campaign. Проверь prompt injection, sandbox escape attempts, tool supply chain, delegated auth, approval spoofing, secret leakage, MCP/A2A boundary и compromised model behavior. Все high-impact actions должны fail closed и оставлять audit.

Промпт 14.6: Eval и capacity qualification

Собери offline qualification suite и проводи repeated experiment против direct/deterministic baseline. Затем выполни load/soak/chaos tests: concurrency, queueing, provider limits, checkpoint store degradation, worker loss и budget exhaustion. Построй capacity model и cost per successful task, не cost per request.

Промпт 14.7: Framework/protocol decision

Используя teardown evidence, защити окончательный build-vs-buy ADR. Собственный runtime остаётся reference implementation; production flow может использовать выбранный framework. Для MCP/A2A/AG-UI integration покажи wire conformance, auth и failure ownership. Удали любой protocol, который не создаёт interoperability value.

Промпт 14.8: Staged rollout

Спроектируй rollout: offline replay, shadow, internal read-only, canary tenants, scoped write actions и gradual expansion. Для стадии задай entry/exit metrics, manual fallback, kill switch, rollback и on-call runbook. Запрети автоматическое расширение permissions по положительной product metric.

Промпт 14.9: Incident game day

Проведи game day из пяти связанных отказов: provider partial outage, poisoned document, duplicate side effect, checkpoint latency и compromised MCP server. Я управляю incident: detect, contain, revoke, degrade, recover, reconcile и preserve evidence. После сделай blameless postmortem и regression tests.

Промпт 14.10: Principal-level defense

Проведи 90-минутную защиту в ролях Architecture, Security, SRE, Applied ML/Evals и Product. Проси показать executable evidence, а не slides. Атакуй assumptions, hidden state, retry semantics, evaluator validity, cost и migration. Заверши списком blockers, follow-ups и решением hire/no-hire для Senior AI Agent Engineer.

Финальные артефакты

1. RFC, C4/data-flow diagrams и ADR log.
2. Собственный framework с test suite и versioned releases.
3. Production implementation и reproducible environment.
4. Model/tool/protocol contracts и golden fixtures.
5. Threat model, red-team report и incident runbook.
6. Versioned eval datasets, graders и release scorecards.
7. OTel traces/dashboards, SLO и capacity/cost model.
8. Demo crash recovery, approval, rollback и safe degradation.

Финальный Gate

- Ни один critical path не зависит от скрытого framework behavior.
 - Система переживает crash, duplicate delivery и provider degradation.
 - Quality/safety/cost подтверждены frozen dataset и repeated trials.
 - High-impact effects ограничены policy, identity, approval и audit.
 - Автор способен объяснить каждую границу и удалить неоправданный agent.
-

Универсальный банк промптов для любой темы

Эти шаблоны добавляются после каждого предметного промпта. Подставляй название механизма, artifact и dataset; так один модуль превращается в полноценный цикл «понял → посмотрел реализацию → проверил → сломал → измерил».

Все команды на создание или изменение кода адресованы ИИ-тьютору. Студент управляет реализацией, читает её и проверяет evidence, но не обязан писать код с пустого файла.

U.1: Создай понятную теорию

Создай или обнови конспект [MECHANISM] в Markdown/PDF. Начни с проблемы и простого примера, затем объясни ключевые понятия, execution flow, ограничения и типичные ошибки. В чате используй одну схему и не более одной таблицы. Не переходи к реализации до команды к практике.

U.2: Спланируй практику

Покажи цель runnable demo, дерево файлов, основные interfaces, ограничения, acceptance criteria и план из небольших шагов. Объясни, что намеренно упрощено. После плана самостоятельно переходи к реализации.

U.3: Реализуй и проверь

Самостоятельно реализуй [MECHANISM] в репозитории, добавь deterministic tests и запусти проверки. Не вставляй полный листинг в чат. Сообщи изменённые файлы, ключевые решения, результаты tests и оставшиеся ограничения.

U.4: Проведи guided code tour

Сначала покажи карту файлов и один end-to-end trace. Затем объясняй по одному или двум компонентам за ответ: вход, состояние, важные ветки, ошибки и tests. После каждого шага задавай один prediction question и жди моего ответа.

U.5: Сломай реализацию

Построй fault matrix для [MECHANISM]: malformed input, timeout, cancellation, duplicate, reordering, partial response, crash, restart, stale state, dependency degradation и version mismatch. Сам добавь deterministic tests, запусти их и покажи trace двух наиболее важных отказов. Попроси меня предсказать результат до запуска одного из tests.

U.6: Атакуй trust boundaries

Составь abuse cases для [MECHANISM] : prompt injection, poisoned data, confused deputy, privilege escalation, cross-tenant access, secret exfiltration и audit tampering. Сам реализуй минимальные security tests. Для каждого риска свяжи control, test, telemetry и residual risk, затем вместе со мной прочитай один security-relevant diff.

U.7: Построй честный benchmark

Создай frozen dataset и минимум два более простых baseline для [MECHANISM] . Задай outcome, trajectory, safety, latency, tokens и cost metrics. Сам запусти repeated trials, покажи uncertainty и threshold практической значимости. Попроси меня принять решение keep/change/remove.

U.8: Сопоставь с framework

Только после from-scratch Gate найди соответствия [MECHANISM] в [FRAMEWORK] : source modules, public abstractions, canonical state, guarantees, extension points и application-owned risks. Сам реализуй небольшой framework-вариант на том же behavioral suite и подтверди актуальную версию официальными источниками.

U.9: Проверь понимание и обнови конспект

Задавай по одному вопросу на чтение кода, prediction, failure semantics и архитектурный trade-off. Если я ошибаюсь, покажи конкретный trace или test, а не давай абстрактную лекцию. После разбора обнови конспект и PDF: добавь реализованные примеры, мои выводы и ссылки на код.

U.10: Проведи итоговый review

Проведи review как skeptical architect. Проверь, могу ли я проследить request, объяснить ключевой код, найти ошибку в небольшом diff, прочитать результаты tests и выбрать между build/buy/remove. Определи, какие agent components можно заменить deterministic code или одним model call. Заверши rubric и конкретными пробелами для повторения.

Рекомендуемый ритм одного модуля

1. **День 1:** теория, конспект и вопросы.
2. **День 2:** план и реализация тьютором с нуля.
3. **День 3:** guided code tour и end-to-end traces.
4. **День 4:** contract/property/fault/security experiments.
5. **День 5:** frozen eval и простые baselines.
6. **День 6:** framework mapping, общий behavioral suite и source teardown.
7. **День 7:** review diff, ADR, обновление конспекта и устная защита.

Если Gate не пройден, следующий модуль не начинается. Если механизм не победил простой baseline и не создаёт необходимую isolation/security boundary, он удаляется.

Критерий Senior AI Agent Engineer 2026

Трек завершён не тогда, когда просмотрены все темы, а когда ты способен:

1. Выбрать deterministic workflow вместо агента и защитить это решение.
 2. Направить ИИ-агента на реализацию agent runtime и durable executor без framework, проверить код и объяснить каждый critical path.
 3. Объяснить и проверить context, retrieval и memory lifecycle.
 4. Ограничить tools/agents identity, policy, sandbox и approval.
 5. Доказать качество через trajectory eval, fault/security suites и статистически честные baselines.
 6. Интегрировать MCP/A2A/AG-UI на wire level и владеть failure semantics.
 7. Выбрать framework по executable evidence и мигрировать без потери correctness.
 8. Вывести bounded autonomy в production с SLO, audit, rollback и incident response.
-

Реестр первичных источников

API и preview-функции меняются быстро. Перед каждым framework/provider prompt необходимо проверить release notes, зафиксировать spec/SDK/model capability и подтвердить пример contract-тестом.

- [OpenAI Function Calling](#) и [Using Tools](#).
- [OpenAI Agents SDK](#).
- [Anthropic Tool Use](#) и [Fine-grained Tool Streaming](#).
- [Gemini Function Calling](#).
- [MCP specification 2025-11-25](#) и [security guidance](#).
- [A2A 1.0 specification](#).
- [AG-UI documentation](#) и [event lifecycle](#).
- [LangGraph documentation](#).
- [LlamaIndex agents/workflows](#).
- [Microsoft Agent Framework](#).
- [CrewAI Flows](#).
- [OpenTelemetry GenAI semantic conventions](#).
- [OWASP Top 10 for Agentic Applications 2026](#).